Dependency Analysis and Plan Obfuscation in Classical Planning

Ayman El-Sayed
George Mason University
aelsaye6@gmu.edu

***Abstract*—Classical task planning has its application in autonomous systems on deciding the actions a robot should take to achieve a goal. Given a fully specified domain, problem, and goal, the agent can determine what to do based on the solver used. However, commonly used solvers are prone to producing plans that reveal intent to a passing observer. This is a concern in multi-agent settings that handle sensitive information, such as hospitals, where staff must respect their patients' privacy. This project develops two complementary capabilities: (1) dependency analysis, which identifies which actions have casual links, and (2) plan obfuscation, which restructures or augments plans to better hide goal-critical actions from an observer. These capabilities are achieved using the technical stack of Python scripting with the Unified Planning and Fast Downward libraries.**

I. Introduction

One of the primary goals behind autonomous systems is to ensure they are capable of making multiple decisions in sequence. While various decision-making algorithms have been developed to fill in this niche, this paper will only focus on one of them: classical task planning. This is a goal-based algorithm that describes *what* an agent wants and does in terms of sequences of actions. Key assumptions of this algorithm include a fully observable and deterministic environment, discrete and fully deterministic robot actions, a single goal for the robot to achieve, and the aim to produce a sequence of actions to achieve to goal [1]. Note that a user need not to be constrained by these assumptions. For example, it is possible to adapt the algorithm to allow an agent to achieve more than a single goal at the end of a plan.

This can be set up through the use of open-source frameworks *Unified Planning*, providing a Python API for modeling automated planning problems by selecting the most appropriate planning engine [2], and *Fast Downward*, which solves planning problems encoded in PDDL using heuristic search [3]. The integration of these libraries facilitates efficient motion planning for multi-agent systems. Furthermore, a novice within the robotics field can achieve proficiency rapidly, provided they have a background in programming.

Still, existing decision-making algorithms and plan resolvers were not designed with security in mind. The combination thereof produces plans with a structure that can reveal intent to an external observer, even when a satisficing (vs. optimal) approach is used. This is not ideal in sensitive settings such as hospitals or military operations, where privacy breaches carry an extensive risk. Respectively, intent-revealing plans may include a delivery robot repeatedly going between a nurse and a specific patient room, or a drone circling a building for a prolonged time before returning to base. These allow an adversarial observer to infer the goal easily (treatments, reconnaissance) then exploit it to predict future routes, influence outcomes, or target highly-valued resources. As such, the produced plans must not rely on a trajectory that leads to obvious intent and, therefore, information leakage.

To prepare for mitigation, I integrated the *Unified Planning* and *Fast Downward* libraries in a Python script with a newly created data structure (a doubly linked list) that allows me to quickly analyze the plan, move actions, and create new ones for the agent. Each action includes metadata describing how it affects the state of the world (e.g., the effects of the action and what is true about the environment before and after it), how it relates to other actions (e.g., `open_door` and `close_door` causally support each other), and which independent sub-goals were achieved, if any. This paves the way for analyzing dependencies between actions to better understand how the produced plan can be modified without violating constraints, and, in turn, to create two plan obfuscation algorithms: *Alternating* and *Diverting*. *Alternating* permutes the order of the subgoals the agent achieves if the plan's final goal can be decomposed into independent subgoals, whereas *Diverting* selects an action at random, inserts another random action that does not violate preconditions, and then generates a new subplan as a detour.

Any concrete examples given will be based on the `hri690` pddl files provided by the Interaction Specification Language (ISL) parser [4]. See Fig. 10 for a visual of the problem.

II. Approach

As already mentioned, the dependency analysis and plan obfuscation algorithms (*Alternating*, *Diverting*) were developed in Python using libraries that assist with plan generation and obtaining relevant information from a given problem and domain. To make replication easier, I opted to use Python's virtual environment to minimize library conflicts in the main system. Under a newly created directory, first create the virtual environment using the following Python command:

```
python -m venv env
```

This will create a new directory called "env" under the current working directory. Next, create the file `requirements.txt` with the list of libraries that will be used for the project:

```
unified-planning
```

```
up-fast-downward
graphviz
```

Note that the `graphviz` library isn't necessary for the algorithms; its only used to generate visuals. Then, activate the virtual environment with the command appropriate for your system [5]:

Platform	Shell	Command
POSIX	bash/zsh	`$ source env/bin/activate`
	fish	`$ source env/bin/activate.fish`
	csh/tsch	`$ source env/bin/activate.csh`
	pwsh	`$ env/bin/Activate.ps1`
Windows	cmd.exe	`env\Scripts\activate.bat`
	PowerShell	`env\Scripts\Activate.ps1`

Finally, install the libraries from the created text file earlier using the following command:

```
pip install -r requirements.txt
```

This completes all steps necessary to prepare the project. If you're cloning from the GitHub repository [6], then `requirements.txt` will be already provided.

A. Generating Fundamental Codebase

Having ready a list of actions the agent needs to complete its goal is a prerequisite to dependency analysis and plan obfuscation. I used ChatGPT to generate the initial source file using the following prompt:

> "I need a classical planner over pddl (problem.pddl and domain.pddl) that generates a plan where action includes a label, pre-condition, and effect (conditions that changed after this action).
>
> - Language: python
> - Libraries: unified-planning, up-fast-downward
>
> The function outputs a tuple:
>
> - First is the current truth, before the first step gets executed.
> - A linked list of nodes where each node contains a label, pre-condition, and effect (conditions that changed after this action).
>
> Example use case for using the output: while traversing a linked list, the program uses the current node and checks if pre-conditions are compatible with current truth. After that, the effects are used to alter the current truth. This is continued until the program hits the end of the linked list, then the current truth now represents the final state of the plan."

This provided most of the functionality I needed, but it had bugs and required refinement through iterative prompting. The generated helper functions were helpful for grounding and testing expressions, but the main program had questionable design decisions, such as using bare nodes (the linked list was not contained) and placing all functions in a single file. This made it difficult to work with, so I eventually redesigned the linked list to be contained (with one exception: generating subplans) and separated the helper functions into different files.

B. Building Linked List from Resolved Plan

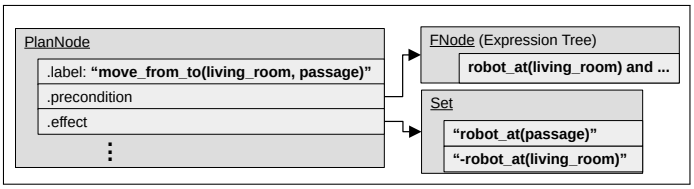


Fig. 1. PlanNode for an action when building the linked list for the first time.

The Python program expects two arguments from the user: the domain file (what actions the agent can do, how these actions affect the world, and how the agent understands the world) and the problem file (initial state of the world and the agent's goal) [1], in that order. Once these files are provided, the program parses the problem using an instance of `PDDLReader` from *Unified Planning*'s io module. The parsed problem is of type `up.model.Problem`, which is saved in a `PlanList` instance as it will help facilitate plan manipulation later.

Even though the parsed problem contains relevant information such as defined actions and initial fluents, no plan has been actually generated yet. I rely on `OneshotPlanner` using the *Fast Downward* planner in order to generate one. This is a convenient API that, as indicated by the name, returns a solution plan (of type `PlanGenerationResult`) in one shot given a problem.

Afterwards, the program is ready to build a linked list. The bulk of this work is done in the `problem_to_plan_list` function. Refer to Fig. 2 for a clearer understanding of how the results from `OneShotPlanner` are used. The general idea is, for each action the agent takes, the `InstantaneousAction` is used as the schema, and bindings are created to map each parameter to *actual* parameters (e.g., if the first parameter is "location from" then it should be mapped to "living_room"). This allows the ability to ground the precondition and effects for each action, resulting in a node like the one in Fig. 1 to be created (refer to Fig. 3 on how grounding is done). This is a partial result since the program hasn't filled other properties of all `PlanNode` at this phase (hidden by the ellipsis in Fig. 1). In fact, it is not yet known how each action affects the world before and after execution, how it contributes toward the final goal, or whether it is supported by or supports other actions. To determine this, the program proceeds to the next phase: dependency analysis.

C. Dependency Analysis

As previously mentioned, the linked list in its current state does not provide sufficient information for plan obfuscation algorithms to leverage. This phase is divided into two subtasks:

1) Replay: On a high level, replaying is simulating the actions the agent takes from the beginning to the end. Before replay, the program has already created a linked list of actions with minimal metadata, plus storing the parsed problem and the initial truths of the environment. The replay functionality

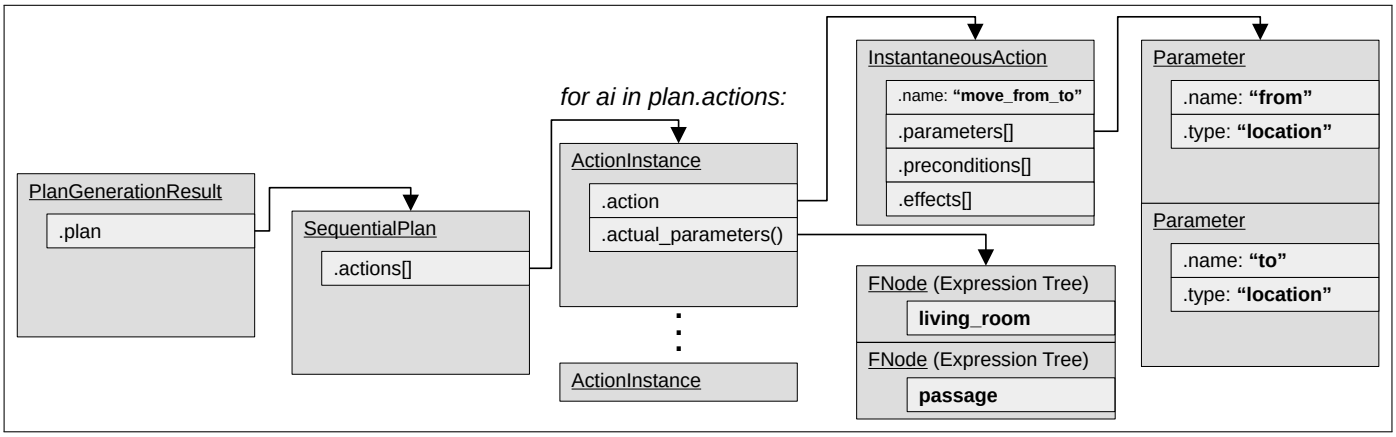


Fig. 2. Breakdown of objects used while building a linked list from a resolved plan.

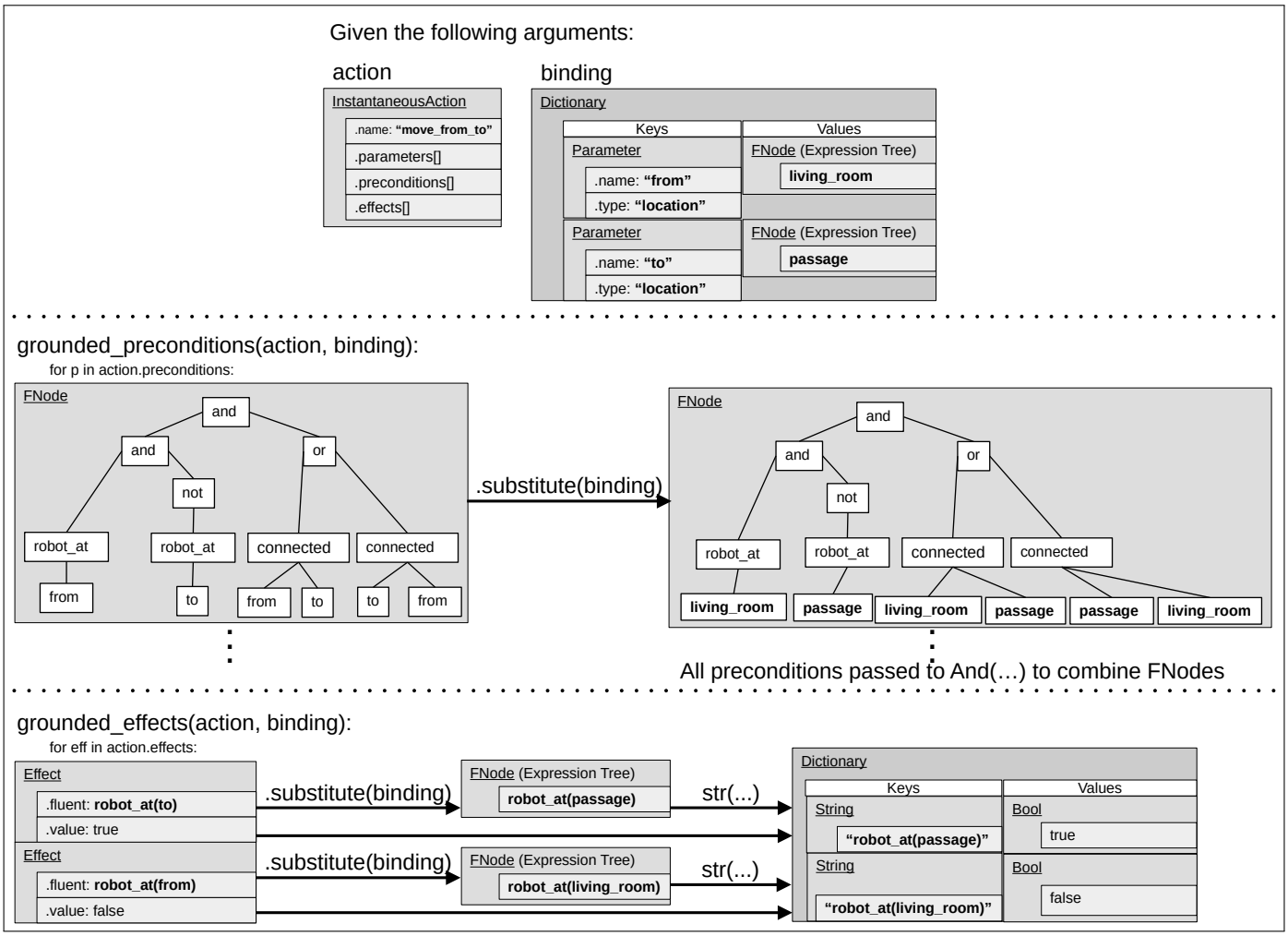


Fig. 3. Details of grounding functions. The substitute method from `FNode` is leveraged to replace generic parameters with actual arguments as defined by the given binding

assumes the initial truths are the truths before the *first* action is executed. For each action, using its already available information per Fig. 1:

- Check that there is no violation of the action's precondi-

tions. This is done by traversing its expression tree and ensuring the fluents appear in the set of truths prior to the execution of the action. If a violation is found, then replay cannot continue and will return false.

- Assign the set of prior truths to the action's `truths_before` property. Duplicate this set for the action's `truths_after` property.
- Modify the action's `truths_after` property (set of truths after execution) based on the deltas contained in the `effect` property. If a predicate in `effect` isn't already in the set of post-execution truths, it has to be added. If a predicate preceded by "-" in `effect` is already in the set of post-execution truths, it needs to be deleted.
- Take the intersection of post-execution truths with the parsed problem's final goal, and add it to the action's `goals_so_far` property.
  - Take the set difference between the current action's and the previous action's `goals_so_far` property, and assign this to current action's `goals_just_added` property. If the resulting set is not empty, then this action inherently contributes toward the goal.
- The truths after execution for this action becomes the truths prior execution for the next action.

These steps are repeated until all actions are validated and have their metadata extended. To summarize, these metadata already existed prior to replay:

- `label` – Full action name with objects passed as arguments. Example: "`move_from_to(living_room, passage)`"
- `precondition` – What predicates need to be true in the environment before execution.
- `effect` – What predicates it modifies in the environment, indicated by delta strings.

After replay, additional metadata has been added:

- `truth_before` – Set of all predicates in the environment before execution of the action.
- `truth_after` – Set of all predicates in the environment after execution of the action.
- `goals_so_far` – Cumulative subgoals toward final goal.
- `goals_just_added` – Any subgoals, if any, that contribute toward final goal.

2) Support Links: Next subtask in dependency analysis. It has already been discussed that an instantaneous action is made possible once its preconditions are satisfied. This may be because the required predicates are already true in the environment, or because a prior action served as a prerequisite by introducing one or more predicates needed to meet those preconditions. The latter is more likely for actions appearing later in the linked list.

The program has an action b of interest. For each action b, the objective is to find if there are any action a that supports b. The first step in creating support links is to extract positive predicates from the preconditions of action b. See Fig. 4 for how this is done. Once this is done, then determine if each predicate was added by a prior action a. A visual provided by Fig. 5 demonstrates this on a high level. To ensure correctness, the program performs additional checks to ensure that no actions between a and b delete the predicate added by a that is needed for b.

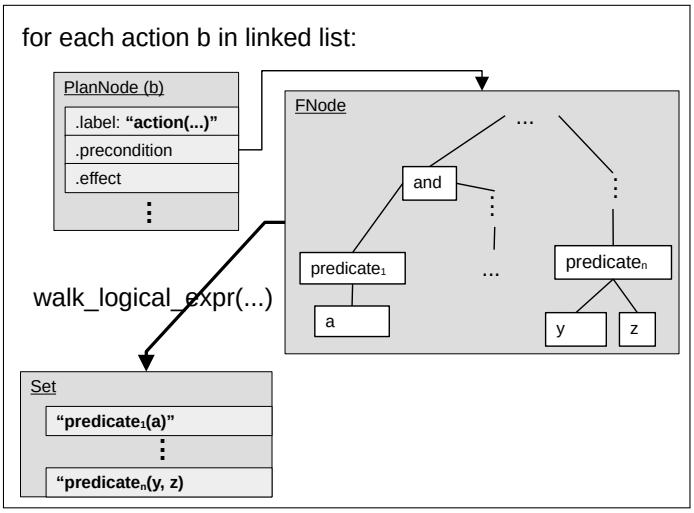


Fig. 4. The precondition from each `PlanNode` is an `FNode` backed by an expression tree, so positive predicates need to be collected through a helper function to obtain an easily enumerable set. `walk_logical_expr` recursively walks the tree.

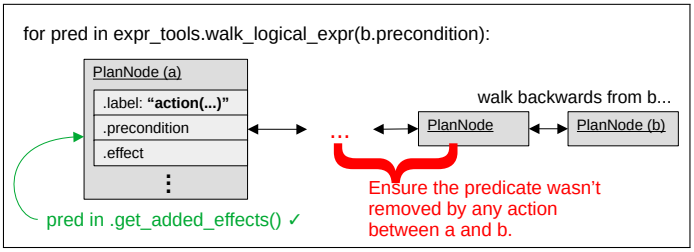


Fig. 5. Given action b, find a supporting action a.

By the end of this subtask, all `PlanNode` actions have their metadata extended to include the following new properties:

- `supports` – This action satisfies preconditions for all actions in this list.
- `supported_by` – This action has its own preconditions satisfied by all actions in this list.

These can be thought of as undirected support edges.

Results: An example of a complete dependency analysis is shown in Fig. 6. The example shows one action can be capable of supporting multiple later actions (e.g., 4–5 and 4–6), not just a single one. Actions highlighted in green have their `goals_just_added` set non-empty. Respectively, these are {`is_open(entry_closet)`}, {`is_open(refrigerator)`}, {`is_open(stove)`}.

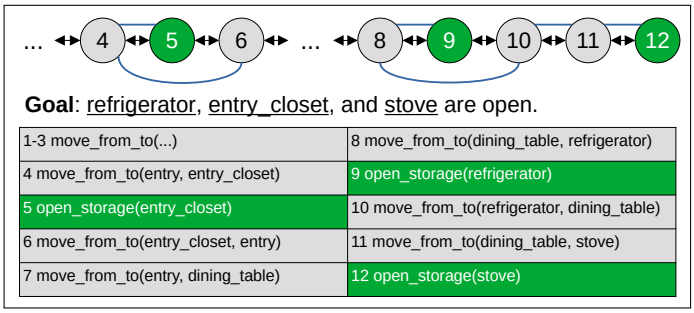


Fig. 6. Dependency analysis for the goal given. Support edges are in blue. Most actions support their direct successor, with the exceptions of 4 & 6 and 8 & 10. Actions highlighted in green contribute toward final goal.

D. Plan Obfuscation

The program moves on to test different obfuscation methods after analyzing dependencies. The obfuscation methods rely on randomness using Python's `random` library. For repeatable results, a seed can be set from the program using the `-s` or `--seed` argument.

1) Alternating: Recall that dependency analysis takes the cumulative subgoals of each action and computes the set difference between adjacent actions to determine whether a given action at an instant contributes to the final goal. If multiple actions are needed to satisfy the final goal, like in Fig. 6, then it is possible to randomize the order in which these subgoals are visited. The key approach is stitching subplans. The algorithm takes this information, plus the parsed problem, to do the following: traverse the linked list, adding all subgoals in order to a temporary Python list. Randomize this list until it no longer matches the original order. Then, for each subgoal:

- Clone the parsed problem. Recall from Section II-B that this was obtained when building the linked list for the first time from the resolved plan.
  - This problem's goal is currently the real overall goal. However, it will be a deliberate decision to set it to the current subgoal instead.
  - It will also be required to set the problem's initial predicates to the final truths of the previous subplan. If this is the first subgoal seen, then the initial truths of the original plan is used instead.
- Use the same approach from Section II-B, except this time the modified problem is fed instead of the problem generated from the original pddl files.
  - Stitch the obtained subplan together with previous resolved subplans.
- Ensure the overall plan to date is still valid by performing checks using methods from Section II-C1. On failure, the algorithm reverts to the original plan.

The final result will be all subplans stitched together (see Fig. 7) that still satisfy the overall goal.

2) Diverting: If a particular order in which the subgoals are visited makes the final goal obvious, then *Alternating* is satisfactory to hide the intent better. However, if that is not enough, then there is also the option of intentionally introducing noise into the plan. *Diverting* achieves this by selecting some action a at random, picking a detour action b (the parsed problems stores all possible actions) as a direct successor, then generating a new subplan. Since the detour action is part of the new subplan, it suffices to discard the old subplan after action a then attach the new subplan as a direct successor to action a. This process can be repeated as many times as necessary.

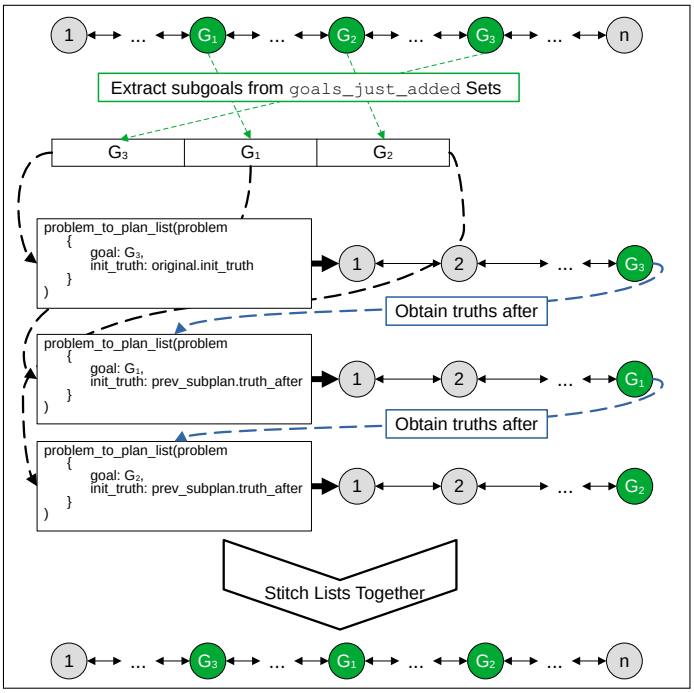


Fig. 7. High-level visualization of *Alternating* obfuscation. The order in which the subgoals are extracted will vary from this diagram due to randomness.

Two main challenges need to be addressed. The selected detour action...

- ... do not bind to actual objects. The parameters only contain types.
- ... must have its precondition satisfied.

This complicates the implementation to an extent.

For the first challenge, *Unified Planning* offers a method to obtain objects based on type. Take Fig. 8 for example. A selected detour action will be of type `InstantaneousAction`, which contains a list of `Parameter` that contains type information. Conveniently, this type information can be passed as an argument to `Problem.objects(type)` to obtain relevant objects. If there are m parameters and n relevant objects, then n^m combinations will be tested for the action. This allows bindings to be created for use in grounding preconditions and effect delta, relying on methods already seen in Fig. 3.

For the second challenge, post-execution truths of selected action a need to be compatible with the detour action's pre-

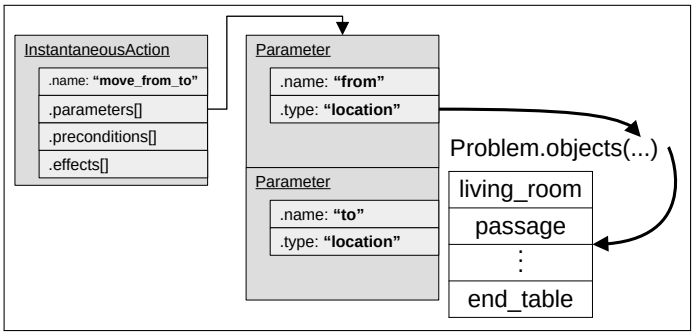


Fig. 8. Obtaining a generator of objects that exist in the parsed problem by passing `Parameter.type` to `Problem.objects(type)` method.

condition. This becomes trivial with grounded preconditions, since the algorithm just needs to traverse its expression tree recursively and compare with the set of action a's post-execution truths.

With these two challenges addressed, the process becomes simple. For each defined action in the parsed problem: enumerate all possible arguments. If preconditions hold for any combination, then calculate post-execution truths of the detour (using grounded effects from earlier) and add it to the list of candidate detour actions (there is an important exception to this: skip candidates that contribute to the final goal since they will make the overall plan shorter instead).

Afterwards, the algorithm will randomly select a candidate. It will need to generate a subplan compatible with the detour, so it uses similar methods from *Alternating* by cloning the parsed problem and modifying it, except the final goal is left intact and only the initial predicates of the cloned problem are set to the candidate's post-execution truths. The generated detour subplan, joined with the initial detour node, is then attached after action a. Refer to Fig. 9 for a visual summary.

III. Discussion

Converting the resolved plan outputted from *Unified Planning* and *Fast Downward* into a doubly linked list of separable actions allowed for the ease of obfuscation. This is evident in *Alternating* where subgoals can be reordered by generating and stitching all sublists together, and evident in *Diverting* where a detour sublist can replace an older sublist. Despite these advances, however, the program is not without flaws. Not only that, but some general approaches had to be discarded either due to time constraint or due to not meeting my expectations.

A. Alternating Issues

A major downside in *Alternating* is that it only collects positive subgoal predicates. For example, if an overall goal is $A \wedge B \wedge \neg C$ then the $\neg C$ can be missed. This *may* cause *Alternating* to create actions with overall goal $A \wedge B \wedge C$ instead, which is undesirable. This is not trivial to mitigate optimally because negations can be nested. One potential approach involves collecting negative predicates N from the overall goal to use alongside positive predicates P. Then, each

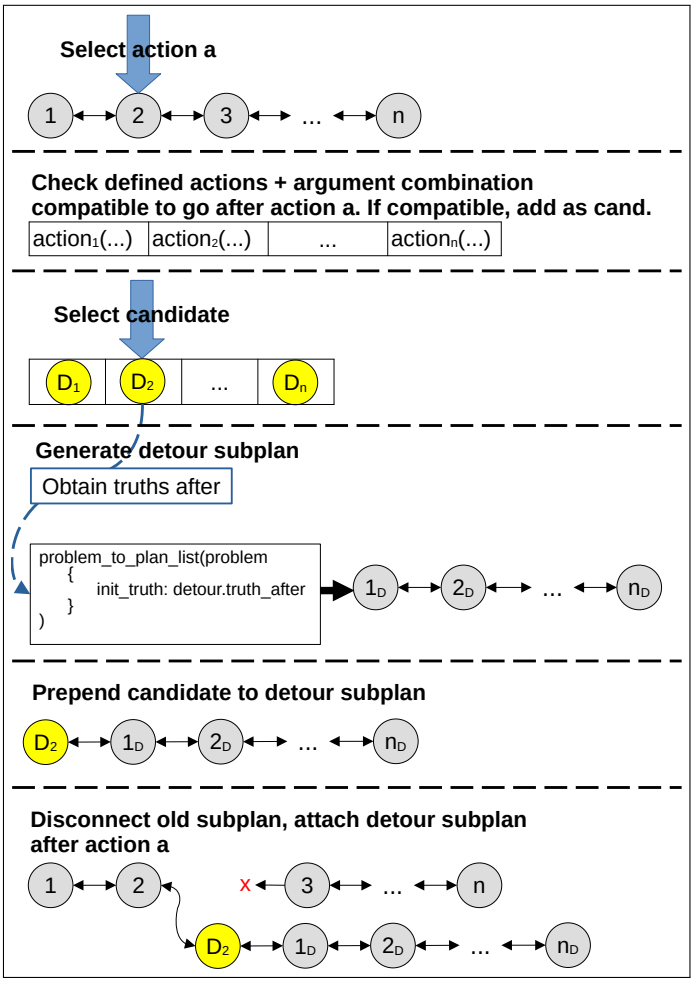


Fig. 9. High-level visualization of *Diverting* obfuscation. **Select action a** and **Select candidate** steps will vary from this diagram due to randomness.

subplan can be created with goal $G = (n_1 \wedge n_2 \wedge ... \wedge n_j) \wedge \text{select}(\{p_1, p_2, ..., p_i\})$, where select randomly picks a p then discards it from P.

The algorithm also does not short-circuit OR expressions. Given $A \vee B$, a plan can be created that satisfies both fluents. A later generated subplan may remove one of these fluents if *Unified Planning* deems it necessary to satisfy preconditions for an action. If this never happens, then detours are technically provided for free.

B. Diverting Issues

This algorithm scales poorly for complex problems. To find detour actions, it must retrieve a actions from the parsed problem. Then, for each action, it iterates through m parameters and retrieves n objects of the corresponding type to test combinations. This results in an average running time of $\Theta\left(aN^M\right)$, which is exponential. This is a sign the search space needs to be optimized.

Despite the linked list having support edges, none of the obfuscation algorithms have used it. One opportunity of optimization is to follow the `supported_by` links, providing

free detour candidates. Compatible objects from the parsed problem still need to be searched, however.

C. Not Using Unified Planning to Its Fullest Extent

Unified Planning has a rich API, including the ability to add actions directly to a parsed problem's resolved plan and to check if the plan is still valid. This may imply my usage of linked lists were unnecessary, but I unfortunately did not have enough time discover all features this library has to offer.

D. LLMs for Code Can Waste Your Time

This project is one of few times I got to explore LLMs (ChatGPT) for assisting with programming. I already mentioned earlier it has been helpful for creating the initial codebase and for creating helper functions for grounding fluents and testing expression trees against a set of predicate strings.

However, only the helper functions needed minimal modifications. The initial codebase was disorganized and primitive, so I had to redo the design to make it easier to reason with.

I also gave it my description of *Alternating* and *Diverting* to assist in implementing the algorithms. Needless to say, ChatGPT's approaches to the algorithms have not been effective. These approaches include adding additional metadata to `PlanNode` like goal depth for use in the algorithms, only for it to not modify the plan most of the time. At some point, I figured it was a red herring and relied on my own intuition instead to create the algorithms.

E. Future Work

This program uses the *Unified Planning* and *Fast Downward* libraries to generate plans from PDDL files. However, as robotics systems increasingly adopt LLMs, a free model such as Ollama could be integrated into this program to support planning. A purely LLM-based approach may produce malformed outputs even with high-quality prompts, so *Unified Planning* should still be used in tandem, resulting in a hybrid approach.

Other obfuscation algorithms exist such as *Multitasking*. The general idea behind it is to select an action achieving multiple goals at once [1]. Due to time constraints and the complexity behind this algorithm, it has been not implemented. It may also be ineffective, but not impossible, for this program since it does not support concurrent plans. This could be integrated once the program is able to support multiple actions occurring at once.

I originally considered integrating a pixel art simulator with this program. This would help visualize the difference between the original and obfuscated plans. Progress on this has stalled due to time constraints and unfamiliarity with Unity and the Robot Operating System (ROS) libraries. To test whether the obfuscation techniques succeed at hiding intent, an empirical study using the pixel art simulator is required with human subjects.

The source code of this program is provided in a GitHub repository [6].

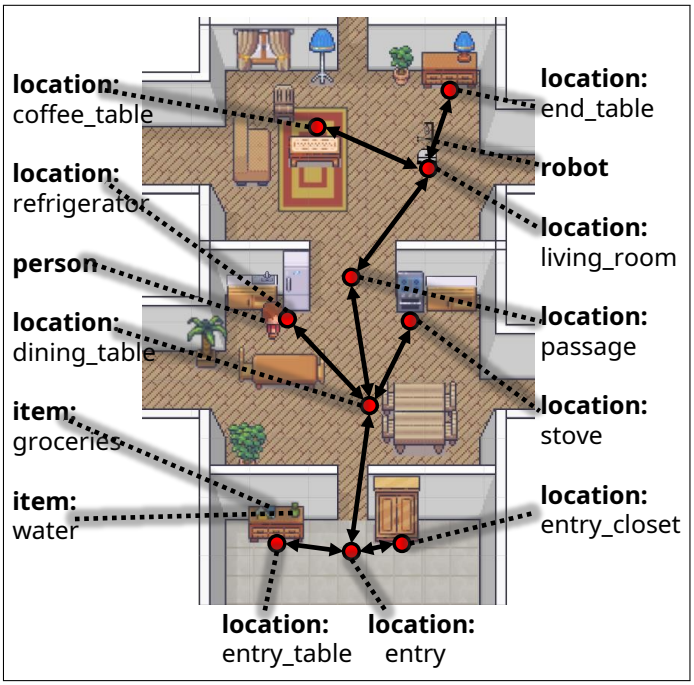


Fig. 10. The environment used when providing concrete examples in other figures.

References

[1] D. Porfirio, "Decision Theory: Humans and Robots," PowerPoint slides, February 2024, CS 690: Human-Robot Interaction, George Mason University.

[2] A. Micheli, A. Bit-Monnot, G. Röger, E. Scala, A. Valentini, L. Framba, A. Rovetta, A. Trapasso, L. Bonassi, A. E. Gerevini, L. Iocchi, F. Ingrand, U. Köckemann, F. Patrizi, A. Saetti, I. Serina, and S. Stock, "Unified planning: Modeling, manipulating and solving ai planning problems in python," *SoftwareX*, vol. 29, p. 102012, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2352711024003820

[3] M. Helmert, "The Fast Downward planning system," *Journal of Artificial Intelligence Research*, vol. 26, pp. 191–246, 2006.

[4] D. Porfirio, M. Roberts, and L. M. Hiatt, "An interaction specification language for robot application development," in *Proceedings of the 2024 ACM/IEEE International Conference on Human-Robot Interaction*, 2025.

[5] Python Software Foundation, *venv — Creation of virtual environments*, 2026, section: How venvs work. [Online]. Available: https://docs.python.org/3/library/venv.html#how-venvs-work

[6] A. El-Sayed, "Dependency Analysis and Plan Obfuscator." [Online]. Available: https://github.com/Ay-A-El/gmuhri_plan_obfuscator